CAMPUSCONNECT

Project Prospectus & Multi-Role Technical Blueprint

DevArambh Hackathon • Google Developer Groups collaboration • 24-hour build

01. Executive summary

CampusConnect converts incomplete student facilities complaints into structured, trackable tickets. Gemini supports interpretation and clarification; deterministic code enforces policy, access, state and persistence.

Problem

Reports arrive through scattered messages and omit location or actionable detail. Students lack progress visibility; staff manually classify, route and reconcile reports.

Solution

Guided intake, structured AI extraction, missing-field clarification, configured routing with evidence, human-reviewed duplicate candidates, ticket lifecycle, event history and operational metrics.

Hackathon thesis

A reliable end-to-end workflow with clear uncertainty handling is stronger than many unfinished features.

02. Source requirements and boundaries

The supplied DevArambh Problem statement describes the CampusConnect minimum requirements. Proposed implementation decisions are distinguished from source requirements.

Requirements in the brief

- Accept written complaint, location and optional photograph.
- Extract and summarise the issue; request missing information.
- Suggest category and responsible department using a supplied routing guide.
- Flag possible duplicates for staff review.
- Staff dashboard for assignment and status updates.
- Show ticket number and progress history.
- One campus; plumbing, electricity, cleanliness and internet connectivity.
- Demonstrate incomplete complaint, clarification, duplicate review and tracking through resolution.

Unknowns

The official routing guide is not yet available. Use replaceable demo rules, clearly labelled as illustrative. Do not invent official departments, policy excerpts, buildings or source references.

Scope guardrail

Hindi entry and recurring-issue analysis are optional. Build them only after the minimum workflow is stable.

03. Product vision and value

CampusConnect is a focused facilities operations product, not a generic chatbot or college super-app.

Student value

- Fast guided reporting; correction of AI interpretation; reference number; progress history; fewer manual follow-ups.

Staff value

- Structured reports; queue filters; assignment and status controls; candidate duplicates with reasons; resolution notes and audit trail.

Admin value

Configurable rule provenance, access management and metrics drillable to underlying incidents.

04. Personas and jobs

Design around the decisions each role needs to make.

Student

Report an issue, correct the interpretation, find the ticket later and understand progress. Prioritise mobile-friendly forms, location selectors and readable history.

Staff

Review actionable work, verify location and route evidence, decide on duplicates, assign ownership and record progress.

Administrator

Maintain routing rules and access, review workload and understand metric definitions.

Security principle

UI visibility is not authorization. The API must enforce role, ownership and scope.

05. Scope and priorities

Use a strict priority ladder for a 24-hour build.

P0 — required

- Complaint, location and optional photo input; AI summary and clarification; four categories; guide-based routing or manual review; persistent ticket number/history; staff assignment/status; student tracking; duplicate review; authentication and deployed app.

P1 — only after P0

- Admin routing interface; recurring-issue analytics; private photo storage; advanced filters and polish.

Non-goals

- Attendance, timetables, food ordering, social feeds, general college chatbot, autonomous maintenance decisions, automatic duplicate merges, predictive failure claims, microservices or Kubernetes.

06. User journeys and acceptance

A screen is not done unless the real API and stored data support it.

Student journey

- Sign in; enter complaint and known location; receive validated draft; answer targeted clarification; confirm/correct; submit; receive unique reference; refresh and retrieve ticket.

Staff journey

- Open queue; inspect summary, location, route evidence and candidates; assign; transition status; record resolution note; see event history.

Duplicate journey

- Retrieve bounded candidates; compare; show reasons; staff confirms/rejects/investigates; preserve original report and reference.

Acceptance standard

Each requirement needs observable pass/fail criteria and a test or repeatable manual verification.

07. Information architecture

Build distinct student, staff and administrator experiences using shared design tokens.

Student pages

- Sign-in; home; report flow; clarification and confirmation; ticket detail and timeline; reopen request.

Staff pages

- Searchable/filterable queue; ticket detail; route evidence; duplicate review; assignment; status and resolution controls.

Admin pages

- Operational overview; routing-rule editor; location directory; controlled role administration.

Design rules

Readable typography, responsive layout, keyboard support, contrast, loading/error/empty states and clear confirmations. Avoid distracting motion.

08. Frontend engineer perspective

The frontend owns presentation and interaction state, not security or authoritative business logic.

Stack

React, TypeScript, Vite, shared API types, reusable status/timeline/form components and a consistent API client.

State

Separate draft state from persisted ticket state. Preserve answers across clarification. Use server responses as canonical state and recover after refresh.

Quality gates

- No fake AI responses presented as live; no trusting browser role claims; no direct client Firestore writes for ticket operations; real button actions; loading/error states; responsive accessible forms.

09. Backend engineer perspective

The backend owns domain rules, trust boundary, orchestration and persistence.

Responsibilities

- Verify Firebase ID tokens; load server-controlled role; validate inputs with Zod; enforce ownership and department scope; call Gemini; validate model output; allocate ticket IDs safely; write events; return typed errors.

API design

Use /api endpoints grouped by role and purpose, with pagination, rate limits for costly AI calls and consistent error envelopes.

Consistency

Use Firestore transactions/batches where appropriate. Never use collection size for ticket IDs. Keep event history append-only where practical and use server timestamps.

10. AI integrator perspective

Gemini is a bounded language-understanding component, not an authority or database writer.

AI tasks

- Extract issue and summary; suggest one of four categories; detect missing fields; ask one targeted question; describe observable photo details; compare new report against bounded candidates.

Structured output

Use supported JSON Schema output and validate again with Zod. Verify category enum and location IDs. Treat model-generated IDs and explanations as untrusted.

Failure behaviour

- Provider timeout preserves draft and permits retry.
- Malformed output is rejected.
- Unknown location prompts user selection or an allowed unknown-location path.
- Missing rule goes to manual review.
- Uncertain duplicate is shown for staff review, never merged automatically.

Prompt injection

Treat complaint text and image text as untrusted data, not instructions. Model cannot change permissions, execute tools or write directly to the database.

11. Routing guide and provenance

Because the official guide is missing, routing rules must be replaceable configuration and demo-labelled.

Rule fields

- Rule ID; category; department ID/name; source title/excerpt/page/section; conditions; active flag; isDemo; created/updated timestamps and actor.

Algorithm

- Validate category; retrieve applicable active rules; choose only configured targets; save rule ID and provenance; show source evidence; if none applies, request manual routing.

Governance

An administrator can replace demo rules without editing prompts or code. Never label a rule official without a verified source. AI cannot invent a department or source excerpt.

12. Duplicate detection

Find candidates deterministically first, then ask Gemini to compare only that bounded set.

Candidate retrieval

Consider category, location, active status, timeframe and text similarity. Limit candidates; do not send the whole database to the model.

Comparison

Require candidate IDs from the supplied list. Explain matches and conflicts; distinguish duplicate from related incident.

Human review

- Pending; confirmed duplicate; not a duplicate; needs investigation.

Integrity

Never automatically merge or delete. Preserve child report and reference. Confirmed duplicate children are excluded from independent-incident counts but remain in audit/report-volume data.

13. Data model and lifecycle

Document Firestore schema and query patterns before parallel development.

Collections

- users/{uid}: profile, role, department and active state.
- campusLocations/{id}: name, type, parent and active state.
- routingRules/{id}: category, department, provenance, demo flag and active state.
- tickets/{id}: public number, reporter, original text, summary, category, location, attachments, status, assignment, canonical incident and timestamps.
- tickets/{id}/events/{id}: actor, event type, previous/new values, comment and timestamp.
- duplicateReviews/{id}: new/candidate ticket, reasons, decision and reviewer.
- counters/{id}: transaction-safe sequence allocation.

Lifecycle

submitted → needs_information → assigned → in_progress → resolved, with a controlled reopened path. Backend code enforces transitions. Students can request reopening but cannot mark resolved.

Queries

Document necessary indexes; paginate queues; avoid unbounded reads; derive metrics from authoritative records.

14. Security and privacy

Security is a release requirement, not a UI feature.

Identity

Verify Firebase ID tokens server-side; read roles from protected data; check ticket ownership; enforce staff scope; restrict admin operations.

Secrets

Local .env is ignored by Git; production Gemini key in Secret Manager; use ADC locally and runtime identity in Cloud Run; never commit service-account keys.

Uploads

Private Cloud Storage; type/size validation; safe object names; short-lived signed URLs or authenticated retrieval; no persistent reliance on container disk.

Privacy

Collect only needed data. Avoid raw complaints, tokens, keys and signed URLs in logs. Confirm campus retention and access expectations before real data use.

15. Cloud Run deployment

Use one containerised service to reduce coordination overhead.

Runtime

React/Vite assets served by Express with /api from the same origin. Node.js 24 LTS, multi-stage Dockerfile, listen on process.env.PORT and 0.0.0.0.

Resources

- Cloud Run; Firestore at deliberately chosen location; private Storage bucket; Secret Manager; dedicated least-privilege runtime service account; Firebase Authentication.

Release steps

- Create/select project and billing; enable APIs; configure Firebase and Firestore; create bucket; configure IAM and secret; deploy with gcloud run deploy --source .; verify URL and end-to-end workflow.

Reliability

Do not assume Cloud Run and data resources share locations. Never seed demo data on production startup. Claim deployment only after testing the deployed URL.

16. Observability and failure recovery

Failures must be visible to developers but safe for users.

Logging

Request IDs, route, duration, outcome and non-sensitive error codes. Never log secrets, tokens, raw complaints or signed URLs.

Useful metrics

- AI success/failure and latency; ticket creation success; clarification completion; invalid transitions; duplicate review volume; API errors and latency.

Failure modes

- Gemini unavailable: preserve draft and allow retry/manual correction.
- Firestore unavailable: no false success; show retryable error.
- Photo upload fails: offer text-only submission if appropriate.
- Permission failure: reject without leaking records.
- Container restart: durable data remains in managed services.

Recovery

Small commits, known-good revision, optional feature kill switches and a deployable vertical slice.

17. Testing and evaluation

Test user outcomes and trust boundaries, not only components.

Test matrix

- Incomplete complaint asks clarification; all four categories accepted; unsupported category rejected; missing route goes manual; same-location candidate reviewed; different-location incident not blindly merged; student cannot read another student's ticket; student cannot resolve; invalid transition rejected; resolution note creates event; invalid upload rejected; malformed AI output handled; AI failure preserves draft; concurrent IDs unique; refresh shows persisted history.

Evaluation metrics

Measure clarification effectiveness, category accuracy on labelled cases, duplicate candidate recall/false positives, routing validity, transition rejection and persistence after refresh. State sample size and conditions; synthetic tests do not prove production performance.

Release gate

Build, typecheck, lint and tests pass; deployed core journey works; student/staff roles are separately verified.

18. Team plan and milestones

Parallelise independent work only after agreeing on shared contracts.

Workstreams

- Frontend: pages, forms, timeline, queue and API integration.
- Backend: auth, schema, ticket lifecycle, events, numbering and permissions.
- AI: extraction, clarification, category and duplicate comparison.
- Integration/deployer: shared contracts, Docker, Cloud Run, tests, seed data and demo.

24-hour milestones

- H0–2: scope, repo, enums, schema and API contract.
- H2–6: skeleton, auth/database/AI smoke tests and first persisted ticket.
- H6–10: clarification, tracking and event history.
- H10–14: staff queue, assignment, status and permissions.
- H14–18: duplicate review and regression.
- H18–21: optional analytics/photo only if P0 stable.
- H21–24: feature freeze, deploy, test matrix and rehearsal.

Rule

Integrate every 2–3 hours; don't defer auth, schema or deployment to the final hours.

19. Risk register

Prioritise risks that can invalidate the demo.

Risks and mitigations

- Missing routing guide → labelled demo rules and manual fallback.
- Gemini key/quota → test early and preserve draft on failure.
- Auth delays → configure Firebase at start; no unprotected production mock.
- Schema drift → shared types and integration owner approval.
- Over-scope → freeze optional features at hour 18–21.
- Cloud IAM errors → test secret and runtime identity early.
- False duplicate → human review and no auto-merge.
- Hallucinated policy → validate output and show provenance.

Kill switches

At H6, drop photo analysis if persistence is broken. At H10, pause analytics if student journey incomplete. At H14, pause polish if staff state transitions fail. At H18, freeze scope.

20. Demo and definition of done

Make the product's reasoning visible and prove persistence.

Demo sequence

- Student submits vague Wi-Fi complaint; AI asks for location; student confirms draft; route evidence appears with demo label where applicable; ticket reference is generated; similar report appears as candidate; staff reviews; staff assigns and resolves with note; student refreshes and sees timeline; admin inspects metrics.

Definition of done

- All minimum requirements implemented; core journey works in deployed app; state/history persist; role boundaries tested; no secrets committed; demo rules labelled; no unsupported policy or prediction claims; build/tests/deployment documented; rollback known.

Pitch

CampusConnect turns scattered campus complaints into validated, explainable, trackable maintenance work—Gemini for understanding, deterministic rules for control, human review for uncertain matches.